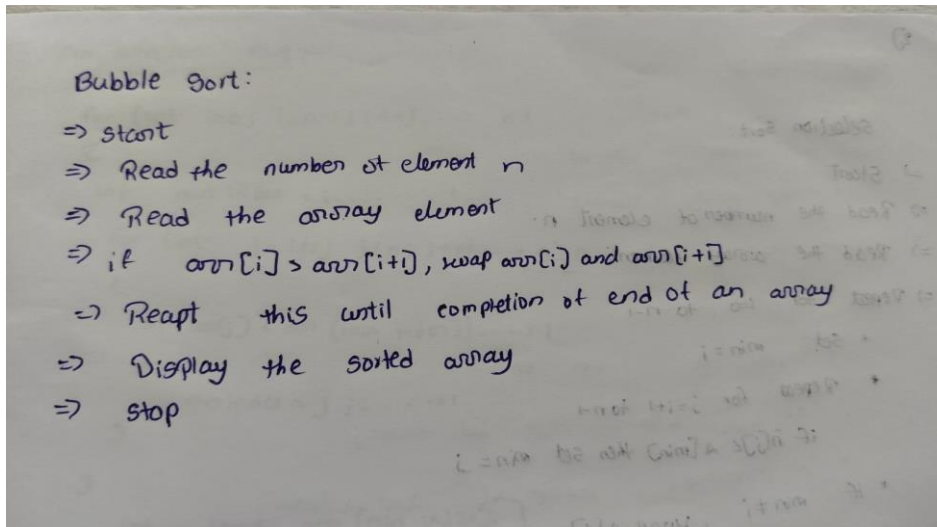


PRACTICAL-01

AIM: To implementation and time analysis of sorting algorithms. Bubble sort, Selection Sort, Insertion sort, Merge sort and Quick sort.

1. BUBBLE SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
}  
  
int main() {  
    int n;  
  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    int arr[n];  
    cout << "Enter array elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
  
    bubbleSort(arr, n);  
  
    cout << "Sorted elements: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    return 0;  
}
```

OUTPUT:

```
Enter number of elements: 5  
Enter array elements: 7  
2  
9  
1  
5  
Sorted elements: 1 2 5 7 9
```

TIME COMPLEXITY:

Bubble Sort

```

for (int i=0; i<n; i++) {
    for (int j=0; j<n-i; j++) {
        if (arr[i] > arr[j]) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}

```

$\Rightarrow$ outer loop : Runs $i=0$ to $n-1$
so it executes $n-1$ times

⇒ inner loop: Pass(i) iterations

$$\begin{array}{ccc} & n-1 & \\ & n-2 & \\ & & \\ (n-1) & + (n-2) & + \dots + 2 + 1 = 1 + 2 + 3 + \dots + (n-1) = \frac{n(n-1)}{2} \\ & & = \frac{n^2 - n}{2} \end{array}$$

$$T(n) = n-1 + \frac{n^2-n}{2} + 1 + 141$$

$$= \frac{2n - \frac{n^2-n+6}{2}}{2} = \frac{n^2-n+6}{2} = n^2/2$$

ignore all constant by asymptotic notation (i.e. $O()$)

$T(n) = O(n^2)$ highest power

worst case:-

condition: Array is in descending order

Process: every comparison leads to swap

comparisons: $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$

$$\frac{n(n-1)}{2} = \frac{n^2 - n}{2} = n^2 - n$$

Average case.

condition: Array element are in random order

Process : About half of the comparisons leads to swap

comparisons: still $\frac{n(n-1)}{2}$ Time complexity $O(n^2)$

Best case:

condition: Array is already sorted

Proble: comparison between bid no wage

Time complexity of n^2 is $O(n^2)$

SELECTION SORT:

ALGORITHM:

selection sort:

- => Start
- => Read the number of element n .
- => Read the array element.
- => Repeat for $i=0$ to $n-1$
 - + set $min=i$
 - * Repeat for $j=i+1$ to $n-1$
 - if $arr[j] < arr[min]$ then set $min=j$
 - * if $min \neq i$, swap $arr[i]$ and $arr[min]$
- => After all passes the array is sorted in ascending order.
- => Display the sorted array
- = stop

PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void selectionSort(int arr[], int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        int minIndex = i;
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (arr[j] < arr[minIndex]) {
```

```
                minIndex = j;
```

```
            }
```

```
        }
```

```
        int temp = arr[minIndex];
```

```
        arr[minIndex] = arr[i];
```

```
        arr[i] = temp;
```

```
    }
```

```
}
```

```
int main() {
```

```
    M.Bhanu Prasad Reddy (924600118970)
```

```
int n;

cout << "Enter number of elements: ";

cin >> n;

int arr[n];

cout << "Enter " << n << " elements: ";

for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

cout << "Original array: ";

for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
    cout << endl;
}

selectionSort(arr, n);

cout << "Sorted array: ";

for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
//cout << endl;
}

return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 9
2
4
6
0
Original array: 9
2
4
6
0
Sorted array: 0 2 4 6 9
```

TIME COMPLEXITY:

Selection sort:-

```
for (int i=0; i<n-1; i++) {
    int minIndex = i;
    for (int j=i+1; j<n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    int temp = arr[minIndex];
    arr[minIndex] = arr[i];
    arr[i] = temp;
}
```

Outer loop: The outer loop runs from $i=0$ to $n-1$. So it executes $n-1$ times.

Inner loop: $\frac{n(n-1)}{2}$

Outer loop	Inner loop	Comparison
1	$n-1$	$(n-1) + (n-2) + (n-3) + \dots + 1$
2	$n-2$	
3	$n-3$	
$\vdots$	$\vdots$	
$n-1$	1	

$$T(n) = \frac{n(n-1)}{2} + n + 1$$

$$= \frac{n^2 + n^2 - n}{2} + n + 1$$

$$= \frac{2n^2 - n}{2} + n + 1$$

$$= n^2 + n$$

$\therefore$ by asymptotic notation and count

$$T(n) = O(n^2) \quad (\therefore \text{highest power})$$

Worst case:-

- Array is in descending order
- For each position the algorithm searches the entire unsorted part to find the minimum
- Comparison $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$
- Swap = $n-1$
- Time complexity = $O(n^2)$

Average case:-

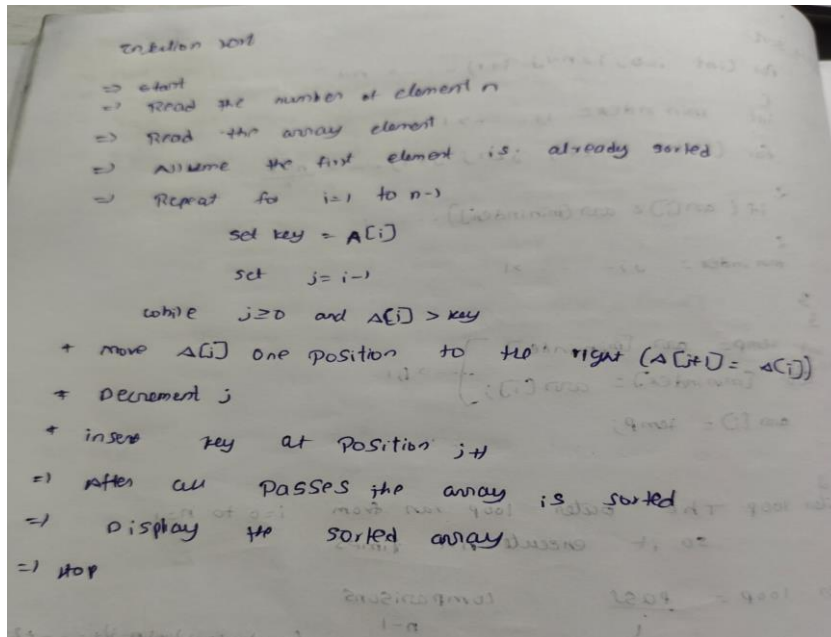
- Array elements are in random order
- Still search the unsorted part fully each position
- Comparison = same as worst case $\frac{n(n-1)}{2}$
- Swap = $n-1$
- Time complexity = $O(n^2)$

Best case:-

- Array is already sorted
- even though no swap are needed algorithm still runs as worst case $\frac{n(n-1)}{2}$
- Swap = 0
- Time complexity = $O(n^2)$

2. INSERTION SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void insertionSort(int arr[], int n) {  
    for (int i = 1; i < n; i++) {  
        int key = arr[i];  
        int j = i - 1;  
  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
        arr[j + 1] = key;  
    }  
}
```

```
}
```

```
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    int arr[n];  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
  
    cout << "Original array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
        cout << endl;  
    }  
    insertionSort(arr, n);  
  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
        //cout << endl;  
    }  
    return 0;  
}
```

OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 7
3
8
2
0
Original array: 7
3
8
2
0
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:

Insertion Sort

```
for (int i=1; i<n; i++)
{
    int key = arr[i];
    int j = i-1;
    while (j >= 0 && arr[j] > key)
    {
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = key;
}
```

1. Best case: Already sorted array
[10, 20, 30, 40, 50] → outer loop runs n-1 times
→ while checks only once for each element
 $T(n) = O(n)$

2) worst case = Reverse sorted array
[50, 40, 30, 20, 10]

each new element must be compared with all previous element and shifted

Pass	Comparison
1	1
2	2
3	3
...	...
n-1	n-1

$1+2+3+\dots+(n-1)$
 $T(n) = \frac{n(n-1)}{2} = O(n^2)$

worst case

- array is in descending order
- each new element must be compared with all previously sorted element and shifted
- comparisons & shift: $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$
- Time complexity = $O(n^2)$

Average case:-

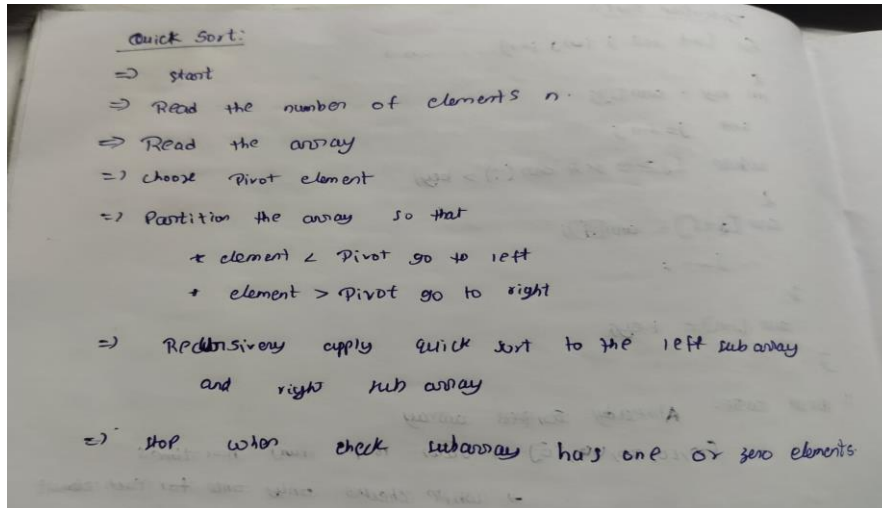
- Array elements are in random order
- on average each element is compared with half of the sorted portion
- comparisons = About $\frac{n(n-1)}{2}$
- Time complexity = $O(n^2)$

Best case:-

- Array is already sorted
- each element is compared only once (no shift needed)
- Time complexity = $O(n)$

3. QUICK SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;

int partition(int arr[], int lb, int ub) { int
    pivot = arr[lb];
    int start = lb; int
    end = ub;

    while (start < end) {
        while (arr[start] <= pivot) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
        }
    }
}
```

```
}

if (start < end) {
    swap(arr[start], arr[end]);
}
}

swap(arr[lb], arr[end]);
return end;
}

void quickSort(int arr[], int lb, int ub) { if
(lb < ub) {
    int loc = partition(arr, lb, ub);
    quickSort(arr, lb, loc - 1); quickSort(arr,
    loc + 1, ub);
}
}

int main() {
    int n;

    cout << "Enter size of array: "; cin
    >> n;

    int arr[n];

    cout << "Enter " << n << " elements: "; for
    (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}
```

```
quickSort(arr, 0, n - 1);

cout << "Sorted array: "; for
(int i = 0; i < n; i++) { cout
    << arr[i] << " ";
}
return 0;
}
```

OUTPUT:

```
Enter size of array: 5
Enter 5 elements: 7
3
8
2
0
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:

Quick Sort:

⇒ The pivot divides the array into two equal halves → $T(n/2)$

→ For sorting we use n element at last

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2T(n/2) + n \rightarrow \textcircled{1}$$

$$T(n) = 2T(n/4) + n/2$$

sub ① in ①

$$T(n) = 2(2T(n/4) + \frac{n}{2}) + n$$

$$T(n) = 2^2 T(n/4) + n \rightarrow \textcircled{2}$$

$n \rightarrow n/4$

$$T(n/4) = 2^2 T(n/8) + \frac{2(n)}{2} \rightarrow \textcircled{3}$$

$$T(n) = 2^4 T(n/16) + \frac{4n}{2} + n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

Apply log both side

$$\log n = \log 2^k$$

$$\log n = k$$

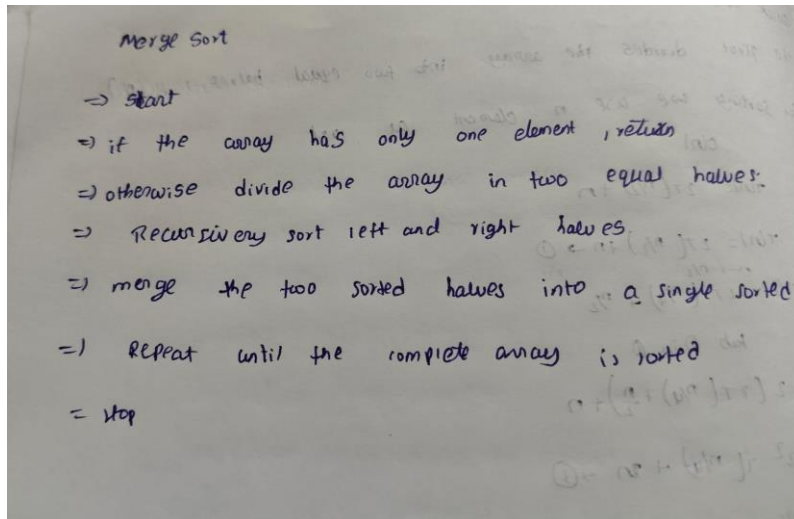
$$T(n) = 2^{\log n} T\left(\frac{n}{2^{\log n}}\right) + \log(n)n$$

$$\therefore a^{\log b} = b^{\log a}$$

$$= n^{\log 2} T\left(\frac{n}{n^{\log 2}}\right) + n \log n$$

4.MERGE SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void merge(int arr[], int left, int mid, int right) { int
```

```
    n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
```

```
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) arr[k++] = L[i++]; else
```

```
        arr[k++] = R[j++];
```

```
    }
```

```
    while (i < n1) arr[k++] = L[i++];
```

```
    while (j < n2) arr[k++] = R[j++];
```

```
}
```

```
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}  
  
int main() { int  
    n;  
    cout << "Enter number of elements: "; cin  
    >> n;  
    int arr[n];  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++) cin >> arr[i];  
  
    mergeSort(arr, 0, n - 1);  
  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++) cout << arr[i] << " ";  
    return 0;  
}
```

OUTPUT:

```
Enter number of elements: 5  
Enter elements: 7  
9  
0  
2  
5  
Sorted array: 0 2 5 7 9
```

TIME COMPLEXITY:

Merge Sort

$\Rightarrow$ The array is divided into two halves repeatedly by number of divisions: $\log_2 n (2T(n/2))$

$\Rightarrow$ merge: each level merge all n elements $O(n)$

Total time $T(n) = 2T(n/2) + n$

$T(n) = 2T(n/2) + n \rightarrow \text{---}$

$n \rightarrow n/2$ put

$T(n) = 2T(n/4) + \frac{n}{2}$

$T(n) = 4T(n/8) + n + n$

$T(n) = 8T(n/16) + 2n$

$n \rightarrow n/4$

$T(n) = 4T(n/16) + \frac{3n}{4}$

$T(n) = 16T(n/16) + n + n$

$T(n) = 16T(n/16) + 4n$

$T(n) = 2^k T(n/2^k) + kn$

$\frac{n}{2^k} = 1$

$n = 2^k$

$\log n$ on both side

$\log n = \log 2^k$

$\log n = k$

$T(n) = 2^{\log n} T(\frac{n}{2^{\log n}}) + n \log n$

$= n^{\log 2} T(\frac{n}{n^{\log 2}}) + n \log n$

$T(n) = n + n \log n$

$(n \log n)$

CONCLUSION:

Bubble Sort and Selection Sort are simple but inefficient with $O(n^2)$ in all major cases. Insertion Sort improves to $O(n)$ in the best case when the array is already sorted. Quick Sort is very efficient on average with $O(n \log n)$, though it can degrade to $O(n^2)$. Merge Sort is consistently $O(n \log n)$ in all cases but requires extra space.